Operating Systems — Assignment 10 Report

Manav Singla, 2402MC02 — xv6 (x86, mit-pdos/xv6-public), qemu-system-i386, 1 CPU — Physical memory accounting, address translation, page protection, page replacement

New system calls (numbers 22–25, wired through `syscall.h`, `syscall.c`, `usys.S`, `user.h`; wrappers in `sysproc.c` read the argument with `argint()` and pass `myproc()->pgdir`): `memstats()`, `vtop(va)`, `pageinfo(va)`, `setreadonly(va)`. The stock `usertests` suite still prints **ALL TESTS PASSED** on the modified kernel.

Q1 — Tracking physical page allocation (`kalloc.c`, `memtest`)

Implementation: four counters were added to the `kmem` struct and are updated under `kmem.lock`: `nalloc` (successful `kalloc()` calls), `nfreed` (`kfree()` calls), `nfree` (pages on the free list) and `npages` (pages managed). At boot, `kinit1/kinit2` build the free list by calling `kfree()` on every page. Those calls would wrongly count as about 57 000 "frees", so `freerange()` sets a `seeding` flag: while it is set, `kfree()` adds to `npages` instead of `nfreed`. `memstats()` takes one snapshot under the lock and prints total allocated, total freed, currently allocated (`nalloc−nfreed`) and available pages, plus the consistency check `npages−nfree`. It returns the number of pages currently allocated.

Output: before allocation: 490 allocated / 214 freed / **276 in use** / 56 789 free, out of 57 065 managed pages (≈223 MB, up to `PHYSTOP`). After `sbrk(64*4096)`, total allocated rises to 554 and in-use to **340 (+64)**, with free pages down by exactly 64. After `sbrk(-64*4096)`, total freed rises to 278 and in-use returns to **276** (PASS). The check value always equals the in-use count, so no page is lost or double-counted. No new page-table page was needed because the heap (0x3000–0x43000) stays inside the first 4 MB page table. `deallocuvm()` frees data pages but never page-table pages.

Q2 — Virtual-to-physical translation (`vm.c`, `vttest`)

Implementation: `walkpgdir()` was made non-static and declared in `defs.h`. `vtop(pgdir, va)` calls `walkpgdir(pgdir, va, 0)`, with `alloc=0` so no page table is ever created. A null result means the PDE is not present (returns −1). A PTE without `PTE_P` returns −2, and a PTE without `PTE_U` (kernel or guard page) returns −3. Otherwise it returns `PTE_ADDR(*pte) | (va & 0xFFF)`: the 20-bit frame address from the PTE combined with the 12-bit page offset. Physical addresses stay below 224 MB, so valid results are never negative.

Output: after `sbrk(3*4096)` at 0x3000, the user program prints the PDX/PTX/offset split and the PA for each address. `0x3000→0x0dfbc000` and `0x3123→0x0dfbc123`: the same frame with the offset preserved (verified in the program). Heap pages 1 and 2 map to 0x0df76000 and 0x0dfbf000, so **contiguous virtual pages are not contiguous physically** because frames come off a LIFO free list. Code, global data and stack pages also translate. For invalid addresses: `0xb000` (above `sz`) gives "PTE not present", `0x40000000` gives "no page table", and both the stack guard page `0x1000` and kernel text `0x80100000` give "PTE_U clear". After `sbrk(-4096)`, the address 0x5000 that translated before becomes "not present".

Q3 — Page protection (`vm.c`, `trap.c`, `pagetest`)

Implementation: `pageinfo(va)` walks the page table and prints PDX/PTX, the raw PTE, the physical frame number (`PTE_ADDR>>12`) and the P/W/U bits. It returns the flag bits, or −1 if the address is unmapped. `setreadonly(va)` clears `PTE_W` and then reloads `CR3` (`lcr3(V2P(pgdir))`) to flush the TLB, because otherwise a cached writable translation could still be used. It only accepts present **user** pages: the kernel half of every page directory points to shared page tables, so clearing W there would affect all processes. In `trap.c`, a new `T_PGFLT` case decodes the error code (bit0 present, bit1 write, bit2 user) for user-mode faults, explains the cause and kills the process. Kernel-mode faults still fall through to the original panic.

Output: the sbrk'd page 0x3000 starts as frame 0xdfbc with `P=1 W=1 U=1` (PTE 0x…067). After `setreadonly`, the PTE becomes 0x…065: only bit 1 (W) changed and the frame is the same. Reads still work, and `setreadonly(0x80100000)` is refused (−1). A forked child inherits the read-only PTE, because `copyuvm()` copies the flags. Its write triggers `PAGE FAULT (trap 14) ... err 7 = [page present | write | user mode]` → protection violation, and it is killed. The parent detects this through a pipe that the child closes without writing, and confirms the byte is still 'X'. The parent's own write then faults the same way. Kernel text (0x80100000) also reports W=0 U=0: xv6 maps it read-only and kernel-only.

Q4 — FIFO, LRU and Optimal page replacement (`pagereplace`)

Implementation: one `simulate(alg, frames)` loop. A hit leaves the frames unchanged. On a fault, an empty frame is used if one exists; otherwise a victim is chosen. **FIFO** uses a round-robin pointer, which always points at the page loaded earliest. **LRU** evicts the frame with the smallest last-use timestamp (timestamps update on every hit and load). **Optimal** evicts the frame whose next reference is farthest in the future, or never comes. Each reference prints the frames, hit/FAULT and the replaced page. Ratios are printed with integer arithmetic because xv6 `printf` has no floats. Usage: `pagereplace [frames] [r1,r2,...]`.

Frames	Algorithm	Faults	Hits	Hit ratio	Fault ratio
3	FIFO	15	5	25.00%	75.00%
3	LRU	12	8	40.00%	60.00%
3	**Optimal**	**9**	**11**	**55.00%**	**45.00%**
4	FIFO	10	10	50.00%	50.00%
4	**LRU**	**8**	**12**	**60.00%**	**40.00%**
4	**Optimal**	**8**	**12**	**60.00%**	**40.00%**
5	FIFO	9	11	55.00%	45.00%
5	**LRU**	**7**	**13**	**65.00%**	**35.00%**
5	**Optimal**	**7**	**13**	**65.00%**	**35.00%**

Observations: with 3 frames the totals are **FIFO 15, LRU 12, Optimal 9**. **Optimal performs best** at every frame count, as it must: no algorithm can beat evicting the page needed farthest in the future. However, it needs the future reference string, so a real OS cannot implement it, and it serves as the lower bound. **LRU is the best practical algorithm here.** It matches Optimal at 4 and 5 frames, because

the string has strong locality (0, 2 and 3 recur), so the recent past predicts the near future well. FIFO evicts by arrival time even when a page is hot. At reference 6 (page 3), FIFO evicts page 0, which was just used at reference 5, and faults on 0 again at reference 7, while LRU evicts the idle page 1 and gets a hit. All three improve as frames are added. With the input `1,2,3,4,1,2,5,1,2,3,4,5`, FIFO goes from 9 faults with 3 frames to **10 with 4 frames (Belady's anomaly)**. LRU and Optimal are stack algorithms and never show this anomaly (10→8 and 7→6).

Summary of modified files

File	Change
`kalloc.c`	Q1 counters in `kmem`, `seeding` flag in `freerange()`, counter updates in `kalloc()` / `kfree()`, new `memstats()`
`vm.c`	`walkpgdir()` made non-static; new `vtop()` (Q2), `pageinfo()` and `setreadonly()` (Q3)
`trap.c`	Q3: `T_PGFLT` case decodes the error code for user-mode page faults and kills the process
`sysproc.c`	`sys_memstats`, `sys_vtop`, `sys_pageinfo`, `sys_setreadonly` wrappers
`syscall.h / syscall.c / usys.S / user.h / defs.h`	syscall numbers 22–25, dispatch table entries, user stubs and prototypes, kernel prototypes
`Makefile`	`i686-elf-` toolchain + warning flags for GCC 16; `_memtest _vttest _pagetest _pagereplace` in `UPROGS`

Fig. 1 — `pagetest`: W bit cleared, then writes by the child and the parent fault (err 7).

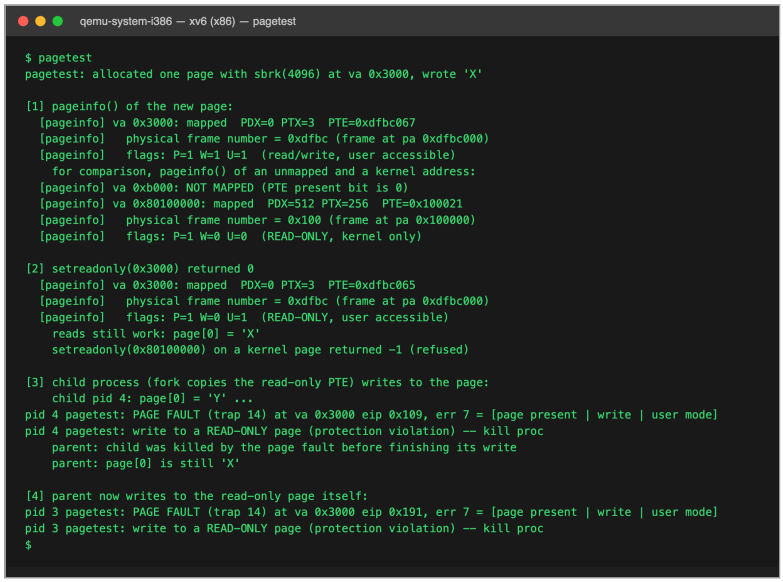


Fig. 2 — `pagereplace` comparison table.

